MODULE 28

Important Components and Modules in LangChain

Lectures Covered

Lec 1 — LangChain Important Components: Models
Lec 2 — Components: Prompts
Lec 3 — Components: Chains and Indexes
Lec 4 — Components: Memory and Agents
Lec 5 — Introduction to Document Loaders
Lec 6 — Text Loaders in Document Loaders
Lec 7 — PDF Loaders in Document Loaders
Lec 8 — DirectoryLoader in Document Loaders
Lec 9 — Text Splitters in LangChain
Lec 10 — WebBaseLoader and CSVLoader
Lec 11 — Length-Based Text Splitter
Lec 12 — Text Structure Based Text Splitter
Lec 13 — Document Structure and Semantic Meaning Based Splitting
Lec 14 — Introduction to Embedding Models
Lec 15 — OpenAI and HuggingFace Embedding Models
Lec 16 — Document Similarity Search with OpenAI Embeddings
Lec 17 — Ollama Embeddings Model
Lec 18 — Getting Started with Vector Stores
Lec 19 — Vector Stores vs Vector Databases and ChromaDB Intro
Lec 20 — FAISS Vector Store
Lec 21 — Working with ChromaDB

SECTION 1: Core LangChain Components (Lectures 1–4)

Lec 1 & 2: Models and Prompts

Models — The LLM Layer

Model Type	Class | Use Case
LLM (text in, text out)	OllamaLLM, HuggingFaceHub | Completion-style generation
Chat Model (messages in/out)	ChatOpenAI, ChatAnthropic, ChatGoogleGenerativeAI | Conversational apps
Embedding Model	OpenAIEmbeddings, HuggingFaceEmbeddings | Vector representations for search

```
from langchain_openai import ChatOpenAI
from langchain_community.llms import Ollama

# Chat model
chat = ChatOpenAI(model='gpt-4o', temperature=0.7)
resp = chat.invoke('What is LangChain?')
print(resp.content)

# Local LLM via Ollama
llm = Ollama(model='llama3')
print(llm.invoke('Explain embeddings in 2 sentences.'))
```

Prompts — PromptTemplate and ChatPromptTemplate

```
from langchain_core.prompts import ChatPromptTemplate, PromptTemplate

# Simple string template
pt = PromptTemplate.from_template('Summarise this text: {text}')
print(pt.invoke({'text': 'LangChain is a framework...'}))

# Chat template with system + human messages
cpt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert in {domain}.'),
    ('human',  '{question}')
])
msgs = cpt.invoke({'domain': 'AI', 'question': 'What is RAG?'})

# Few-shot prompt
from langchain_core.prompts import FewShotPromptTemplate
examples = [
    {'input': 'happy',   'output': 'sad'},
    {'input': 'tall',    'output': 'short'},
]
```

Lec 3 & 4: Chains, Indexes, Memory and Agents

Chains — LCEL Pipelines

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

llm    = ChatOpenAI(model='gpt-4o')
prompt = ChatPromptTemplate.from_template('Answer: {question}')
parser = StrOutputParser()

chain  = prompt | llm | parser          # LCEL pipe syntax
result = chain.invoke({'question': 'What is FAISS?'})

# JsonOutputParser — get structured output
json_chain = prompt | llm | JsonOutputParser()
```

Memory Types

Memory Class	Behaviour | Best For
ConversationBufferMemory	Stores ALL messages — grows unbounded | Short conversations
ConversationBufferWindowMemory	Keeps last K turns only | Long chats with fixed window
ConversationSummaryMemory	Summarises old turns to compress | Very long conversations
VectorStoreRetrieverMemory	Retrieves most relevant past messages via embedding | Semantic memory

Agents and Tools

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain_community.tools import WikipediaQueryRun, DuckDuckGoSearchRun
from langchain_core.prompts import PromptTemplate

tools = [WikipediaQueryRun(), DuckDuckGoSearchRun()]

# ReAct agent: Reason + Act loop
# LLM decides which tool to use, calls it, observes result, repeats
agent  = create_react_agent(llm, tools, prompt)
runner = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = runner.invoke({'input': 'Who invented the Transformer architecture?'})
```

SECTION 2: Document Loaders (Lectures 5–10)

Lec 5 & 6: Document Loaders Introduction and Text Loaders

Document Loaders import data from external sources into LangChain Document objects — each containing page_content (text) and metadata (source, page, etc.).

```
# Every loader returns a list of Document objects
# Document = { page_content: str, metadata: dict }

# Text file loader
from langchain_community.document_loaders import TextLoader
loader = TextLoader('data/notes.txt', encoding='utf-8')
docs   = loader.load()        # list of Documents
print(docs[0].page_content[:200])
print(docs[0].metadata)       # {'source': 'data/notes.txt'}

# Lazy loading (memory efficient for large files)
for doc in loader.lazy_load():
    process(doc)
```

Loader	Source | Install
TextLoader	Plain .txt files | langchain-community
PyPDFLoader	PDF files (PyPDF2) | pip install pypdf
WebBaseLoader	Web pages via URL | pip install beautifulsoup4
CSVLoader	CSV files | langchain-community
DirectoryLoader	All files in a folder | langchain-community
UnstructuredLoader	Word, PPT, HTML, email | pip install unstructured
ArxivLoader	Arxiv papers by ID | pip install arxiv
WikipediaLoader	Wikipedia articles | pip install wikipedia

Lec 7 & 8: PDF and Directory Loaders

```
# PDF Loader
from langchain_community.document_loaders import PyPDFLoader
loader = PyPDFLoader('data/report.pdf')
pages  = loader.load()        # one Document per page
print(f'Pages: {len(pages)}')
```

```
print(pages[0].metadata)     # {'source': 'data/report.pdf', 'page': 0}

# Load all PDFs in a folder
from langchain_community.document_loaders import DirectoryLoader
loader = DirectoryLoader(
    path      = 'data/',
    glob      = '**/*.pdf',   # pattern: all PDFs recursively
    loader_cls= PyPDFLoader,
    show_progress=True
)
all_docs = loader.load()
print(f'Total documents: {len(all_docs)}')
```

Lec 9 & 10: Text Splitters, WebBaseLoader and CSVLoader

```
# Web page loader
from langchain_community.document_loaders import WebBaseLoader
import bs4

loader = WebBaseLoader(
    web_paths = ['https://lilianweng.github.io/posts/2023-06-23-agent/'],
    bs_kwargs  = dict(parse_only=bs4.SoupStrainer(class_=('post-content',)))
)
docs = loader.load()

# CSV loader
from langchain_community.document_loaders.csv_loader import CSVLoader
loader = CSVLoader(file_path='data/employees.csv',
                   source_column='name')   # metadata source field
docs   = loader.load()   # one Document per CSV row
```

SECTION 3: Text Splitters (Lectures 9, 11–13)

Lec 11 & 12: Length-Based and Structure-Based Splitters

Raw documents are too large for an LLM context window — text splitters break them into overlapping chunks that preserve context at boundaries.

Splitter	Strategy | Best For
CharacterTextSplitter	Split at a specific character (\n\n by default) | Simple text, quick baseline
RecursiveCharacterTextSplitter	Try [\n\n, \n, ' ', ''] recursively — keeps paragraphs/sentences intact | Most text (recommended default)
TokenTextSplitter	Split by token count (not characters) | Precise context window management
SentenceTransformersTokenTextSplitter	Token split aligned to embedding model limits | Embedding pipelines

```
from langchain.text_splitter import (
    CharacterTextSplitter, RecursiveCharacterTextSplitter,
    TokenTextSplitter
)

# RecursiveCharacterTextSplitter — recommended default
splitter = RecursiveCharacterTextSplitter(
    chunk_size    = 1000,   # max chars per chunk
    chunk_overlap = 200,    # overlap to preserve context at boundaries
    separators    = ['\n\n', '\n', ' ', '']  # try in order
)
chunks = splitter.split_documents(docs)   # list of Documents
print(f'Chunks: {len(chunks)}  | First chunk: {chunks[0].page_content[:100]}')

# TokenTextSplitter — split by token count
tok_splitter = TokenTextSplitter(chunk_size=512, chunk_overlap=50)
tok_chunks   = tok_splitter.split_documents(docs)
```

Lec 13: Document Structure and Semantic Splitting

```
# Markdown-aware splitter — respects headers
from langchain.text_splitter import MarkdownHeaderTextSplitter
headers_to_split = [('#','H1'), ('##','H2'), ('###','H3')]
md_splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers_to_split)
md_docs = md_splitter.split_text(markdown_text)
# Each chunk carries header info in metadata

# HTML-aware splitter
from langchain.text_splitter import HTMLHeaderTextSplitter
html_splitter = HTMLHeaderTextSplitter(headers_to_split_on=[('h1','H1'),('h2','H2')])

# Semantic splitter — splits at meaning boundaries using embeddings
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings
sem_splitter = SemanticChunker(
    OpenAIEmbeddings(),
    breakpoint_threshold_type='percentile'   # split where similarity drops
)
sem_chunks = sem_splitter.split_documents(docs)
```

Splitter selection guide

Default choice: RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200).
Structured docs (Markdown, HTML): use structure-aware splitters to keep sections together.
Precision needed (fit exact context window): TokenTextSplitter.
Best RAG quality: SemanticChunker — splits at natural meaning boundaries.
chunk_overlap ensures a sentence cut at a boundary still appears in the next chunk.

SECTION 4: Embedding Models (Lectures 14–17)

Lec 14 & 15: Embedding Models — OpenAI and HuggingFace

Embedding models convert text into dense numerical vectors. Semantically similar texts produce vectors with high cosine similarity — the foundation of semantic search and RAG.

Model	Provider | Dims | Notes
text-embedding-3-small	OpenAI | 1536 | Fast, cheap, good quality
text-embedding-3-large	OpenAI | 3072 | Best OpenAI quality
all-MiniLM-L6-v2	HuggingFace | 384 | Fast local, great for RAG
all-mpnet-base-v2	HuggingFace | 768 | Higher quality, slower
nomic-embed-text	Ollama | 768 | Local, no API key needed

```
# OpenAI Embeddings
from langchain_openai import OpenAIEmbeddings
embed = OpenAIEmbeddings(model='text-embedding-3-small')

# Embed a single query
vec = embed.embed_query('What is RAG?')
print(f'Dim: {len(vec)}')   # 1536

# Embed a list of documents
vecs = embed.embed_documents(['LangChain is a framework', 'FAISS is a vector store'])

# HuggingFace Embeddings (local — no API key)
from langchain_community.embeddings import HuggingFaceEmbeddings
hf_embed = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
vec2 = hf_embed.embed_query('Explain transformers')
print(f'Dim: {len(vec2)}')  # 384
```

Lec 16 & 17: Similarity Search and Ollama Embeddings

```
# Document similarity search using embeddings
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

docs_text = ['Python is a programming language.',
             'Deep learning uses neural networks.',
             'LangChain helps build LLM apps.']
query     = 'What tools help build AI applications?'

doc_vecs   = embed.embed_documents(docs_text)
query_vec  = embed.embed_query(query)

sims = cosine_similarity([query_vec], doc_vecs)[0]
best = docs_text[np.argmax(sims)]
print(f'Most similar: {best}')  # 'LangChain helps build LLM apps.'

# Ollama local embeddings — no API cost
from langchain_community.embeddings import OllamaEmbeddings
ollama_embed = OllamaEmbeddings(model='nomic-embed-text')
vec3 = ollama_embed.embed_query('What is a vector store?')
```

SECTION 5: Vector Stores (Lectures 18–21)

Lec 18 & 19: Vector Stores vs Vector Databases

A vector store holds document embeddings and supports fast similarity search. A vector database adds production features (persistence, filtering, scaling, access control).

Feature	Vector Store (in-memory) | Vector Database (production)
Persistence	RAM only — lost on restart | Disk/cloud — survives restarts
Scale	Millions of vectors max | Billions of vectors
Filtering	Limited metadata filter | Rich filtering + hybrid search
Examples	FAISS, in-memory Chroma | Pinecone, Weaviate, Qdrant, PGVector
Best For	Prototyping, notebooks | Production RAG applications

RAG pipeline with vector store

1. Load documents (DocumentLoader)
2. Split into chunks (TextSplitter)
3. Embed chunks (EmbeddingModel)
4. Store embeddings (VectorStore)
5. At query time: embed query -> similarity search -> retrieve top-k chunks
6. Inject chunks into prompt -> LLM generates answer

Lec 20: FAISS Vector Store

FAISS (Facebook AI Similarity Search) is a highly optimised in-memory vector store. It is the fastest option for local prototyping and small-to-medium scale RAG.

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load and split
loader   = PyPDFLoader('data/ml_textbook.pdf')
docs     = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks   = splitter.split_documents(docs)

# Create FAISS vector store
embed  = OpenAIEmbeddings(model='text-embedding-3-small')
faiss_db = FAISS.from_documents(chunks, embed)
```

```
# Similarity search
results = faiss_db.similarity_search('What is gradient descent?', k=3)
for r in results:
    print(r.page_content[:150])

# Save and reload (persist to disk)
faiss_db.save_local('faiss_index')
faiss_db = FAISS.load_local('faiss_index', embed,
                            allow_dangerous_deserialization=True)

# Use as retriever in a RAG chain
retriever = faiss_db.as_retriever(search_kwargs={'k': 4})
```

Lec 21: Working with ChromaDB

ChromaDB is an open-source vector database with disk persistence built-in. Unlike FAISS, Chroma survives restarts and supports metadata filtering — making it ideal for production RAG applications.

```
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

embed = OpenAIEmbeddings(model='text-embedding-3-small')

# Create and persist ChromaDB
chroma_db = Chroma.from_documents(
    documents   = chunks,
    embedding   = embed,
    persist_directory = 'chroma_db'   # saved to disk automatically
)

# Load existing ChromaDB from disk
chroma_db = Chroma(
    persist_directory = 'chroma_db',
    embedding_function= embed
)

# Similarity search with score
results = chroma_db.similarity_search_with_score('Explain attention mechanism', k=3)
for doc, score in results:
    print(f'Score: {score:.4f}  |  {doc.page_content[:100]}')

# Metadata filtering
results = chroma_db.similarity_search(
    'neural networks',
    k=3,
    filter={'source': 'data/ml_textbook.pdf'}
)

# Full RAG chain with ChromaDB
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI

retriever = chroma_db.as_retriever(search_kwargs={'k': 4})
llm       = ChatOpenAI(model='gpt-4o')
prompt    = ChatPromptTemplate.from_template(
    'Answer based only on context:\n\n{context}\n\nQuestion: {question}'
)

def format_docs(docs):
    return '\n\n'.join(d.page_content for d in docs)

rag_chain = (
    {'context': retriever | format_docs, 'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
answer = rag_chain.invoke('What is backpropagation?')
print(answer)
```

	FAISS	ChromaDB
Persistence	Manual save_local()/load_local() | Automatic via persist_directory	
Metadata filter	Limited | Full filter dict support	
Setup	pip install faiss-cpu | pip install chromadb	
Best for	Fast prototyping, notebooks | Production RAG, persistent store	
Scaling	In-memory only | Disk-backed, handles larger datasets	

End of Module 28 Notes — Generative AI Course